Solver Trace Ablation: Diagnosing Reward Hacking and Goodhart's Law in GRPO-Trained Formal Reasoners

Anonymous

Abstract

GRPO with execution-only rewards trains neural formal reasoners to high prove rates while producing structurally degenerate outputs—a form of reward hacking invisible to execution-based evaluation. We propose *Solver Trace Ablation* (STA), a diagnostic protocol that derives an *Intensional Structural Reward* (ISR) via leave-one-out component necessity testing on solver inputs. We establish three findings. **(1)** ISR detects reward hacking that prove rate misses: exec-only GRPO achieves 93% prove rate while ISR collapses from 0.64 to 0.003 over training. **(2)** When used as a reward signal, ISR exhibits Goodhart's Law: low-dose ISR reward (β=0.1) yields 2.96× structural improvement over matched random control (p=0.002) without harming prove rate, while high-dose (β=0.3) is actively worse than random noise. **(3)** The STA protocol generalizes across three formal languages (FOL/Prover9, Prolog/SWI-Prolog, SMT-LIB/Z3), with 4 of 5 identified hacking modes shared across logics and consistent ∼3× ISR improvement at β=0.1. Across 5 reward conditions, 13 seeds, and 3 logic types, we identify 5 distinct hacking strategies and document ISR's transition from useful diagnostic to harmful optimization target.

1 Introduction

Neural formal reasoning systems trained with reinforcement learning from execution feedback have achieved impressive solve rates on logical formalization tasks. GRPO (**?**) with binary execution rewards (proved/not-proved) is a dominant paradigm: the solver provides a ground-truth reward signal, and the model learns to produce formalizations that pass verification.

However, high prove rate does not imply structural soundness. A formalization can "prove" a conclusion by embedding the conclusion as a premise, by repeating effective components to pad length, or by exploiting tautological shortcuts—all achieving r_{exec}=1 while producing logically vacuous proofs. We term these failure modes *reward hacking*: the model satisfies the reward function without solving the intended task.

This paper makes three contributions:

(1) ISR as Diagnostic. We introduce *Solver Trace Ablation* (STA), which applies leave-one-out (LOO) ablation to solver inputs to compute ISR—a measure of component necessity. ISR quantifies structural degeneration invisible to prove rate: in exec-only GRPO, prove rate reaches 93% while ISR collapses from 0.64 (100 steps) to 0.13 (500 steps) to 0.003 (1000 steps).

(2) ISR as Reward Signal: Goodhart's Law. We conduct the first systematic study of ISR as a reward shaping signal. At low dose (β=0.1), ISR reward provides genuine structural guidance: mean ISR is 2.96× higher than a matched random control across 3 seeds (p=0.002), with no significant prove rate difference (p=0.52). At high dose (β=0.3), ISR reward triggers Goodhart collapse: prove rate drops to 18%, worse than matched random noise (48%). This constitutes a clean empirical demonstration of Goodhart's Law in reward shaping.

(3) Cross-Logic Generality. The STA protocol transfers to Prolog/SWI-Prolog and SMT-LIB/Z3 without modification of the core LOO principle. Exec-only GRPO produces reward hacking in all three logics (hacking rates 68–84%), with 4 of 5 hacking modes shared across FOL and Prolog. The β=0.1 sweet spot repli-

cates: 2.99× ISR improvement in Prolog and 2.67× in SMT-LIB.

2 Related Work

Reward Hacking in RL. Reward hacking—where agents exploit reward function gaps rather than solving the intended task—is well-documented in RL (**??**). **?** show that overoptimization of reward models degrades true performance, a phenomenon formalized as Goodhart's Law (**?**). **?** provide a taxonomy of Goodhart effects (regressional, extremal, causal, adversarial), of which our observations correspond to the regressional and extremal categories. **?** study Goodhart's Law in RLHF. Our work provides the first systematic empirical study of Goodhart's Law specifically in formal reasoning reward shaping.

GRPO and Formal Reasoning. GRPO (**?**) uses group-relative advantage estimation for RL without a critic model. Recent work applies GRPO to code generation (**?**), mathematical reasoning (**?**), and formal theorem proving (**?**). **?** apply hand-crafted Prolog-specific structure penalties. EVOM (**?**) uses scalar execution rewards only. None systematically study reward hacking in formal reasoning GRPO or provide diagnostic tools for detecting it.

Formal Logic Datasets. FOLIO (**?**) provides natural language premises with FOL annotations. LeanDojo (**?**) and Baldur (**?**) target theorem proving in Lean. ProofNet (**?**) provides formal mathematics benchmarks. Our work uses FOLIO as the primary testbed and extends to Prolog and SMT-LIB formalization tasks.

Feature Attribution and Ablation. LOO ablation is a standard technique in feature attribution (**?**). SHAP (**?**) provides a principled framework for feature importance. We apply the LOO principle not to neural network features but to *solver input components*, measuring their necessity for proof success. We explicitly acknowledge that ISR is an application of LOO ablation; our contribution is its use as a diagnostic for formal reasoning reward hacking.

Mutation Testing. Our approach is inspired by mutation testing's RIP framework (**??**): a test suite's quality is measured by its ability to detect mutants. We operationalize the *Propagation* condition—whether removing a component propagates to change the solver's verdict—as our structural integrity measure.

3 Method: Solver Trace Ablation

3.1 Problem Setting

Given a natural language reasoning problem, a neural formalizer π_θ generates a formalization $F = (c_1, c_2, \ldots, c_n)$ consisting of n components (premises, axioms, rules, or constraints depending on the logic type). A deterministic solver $\mathcal{S}$ evaluates F and returns a binary verdict $\mathcal{S}(F) \in \{0, 1\}$. Standard GRPO training uses the execution reward $r_{\text{exec}} = \mathcal{S}(F)$.

3.2 ISR: Intensional Structural Reward

For a proved formalization ($\mathcal{S}(F) = 1$), STA performs LOO ablation: for each component c_i, it evaluates $\mathcal{S}(F \setminus \{c_i\})$. Component c_i is *necessary* if removing it causes proof failure:

$$\text{nec}(c_i) = \mathbf{1}[\mathcal{S}(F \setminus \{c_i\}) = 0] \tag{1}$$

ISR is the fraction of necessary components:

$$\text{ISR}(F) = \frac{1}{n} \sum_{i=1}^{n} \text{nec}(c_i) \tag{2}$$

For unproved formalizations ($\mathcal{S}(F) = 0$), $\text{ISR}(F) = 0$ by convention. ISR ranges from 0 (all components are redundant or formalization is unproved) to 1 (every component is individually necessary for the proof).

3.3 Computational Complexity

Full STA requires $O(n)$ solver calls per formalization (one per component). We evaluated an $O(1)$ approximation (`fast_isr`) that tests only the first component. As we show in Section 5.3, this approximation fails: its correlation with full ISR is only ρ=0.18, because hacking patterns (e.g., conclusion embedding) do not necessarily place the exploited component first. The $O(n)$ computation is therefore a necessary cost.

3.4 ISR as Reward Signal

To integrate ISR into GRPO training, we use a multiplicative bonus:

$$r = r_{\text{exec}} \cdot (1 + \beta \cdot \text{ISR}(F)) \tag{3}$$

where β controls the strength of the structural signal. This formulation ensures that ISR only modulates reward for proved formalizations (r_{exec}=1), avoiding penalties for unproved outputs.

3.5 ISR Capability Boundary

ISR measures *syntactic component necessity*—whether each component is individually required for the proof to succeed. It does *not* measure *semantic faithfulness*—whether the formalization correctly captures the meaning of the natural language input. Downstream NLI evaluation yields Spearman $\rho(\text{ISR}, \text{NLI accuracy}) = -0.049$ (p=0.187, not significant), confirming that ISR and semantic quality are orthogonal. This boundary is important for practitioners: ISR diagnoses structural degeneration, not semantic errors.

4 Experimental Setup

Model. Qwen2.5-0.5B-Instruct as the primary model; Qwen3-8B for scaling experiments (Section 8.1).

Data. FOLIO (**?**) with 500 training problems for FOL experiments; equivalent-size Prolog and SMT-LIB formalization datasets for cross-logic experiments.

Training. GRPO with group size K=8, 500 training steps (unless noted otherwise), using Prover9 (FOL), SWI-Prolog, and Z3 (SMT-LIB) as solvers.

Evaluation. Held-out evaluation set. Metrics: prove rate (PR), mean ISR, proved-only ISR (conditioned on r_{exec}=1), mean component count, and genuine rate (fraction of proved formalizations with ISR > 0.3).

Seeds. Three seeds (s42, s43, s44) for replication experiments; single seed for exploratory experiments.

5 Claim 1: ISR as Reward Hacking Diagnostic

5.1 ISR Collapse Trajectory

Exec-only GRPO achieves high prove rate but undergoes progressive structural collapse, as revealed by ISR:

Table 1: ISR collapse over training in exec-only GRPO (FOL, seed s42). Prove rate increases monotonically while ISR collapses.

Steps	PR	Proved-only ISR	Components
100	0.32	0.642	—
500	0.93	0.132	5.58
1000	0.96	0.003	18.55

At 100 steps, the model produces few successful formalizations but those that succeed are structurally sound (ISR = 0.64). By 500 steps, prove rate reaches 93% but ISR drops to 0.13—most components in proved formalizations are unnecessary. By 1000 steps, the collapse is near-total: ISR = 0.003, with 94.8% of proved samples having ISR=0. Component count inflates from 5.58 to 18.55, indicating explosion-style proof padding.

Three seeds at 500 steps reveal seed-dependent hacking modes: s42 exhibits conclusion embedding (ISR = 0.123), s43 exhibits repetition collapse (PR = 13%, ISR = 0), and s44 exhibits tautological collapse (PR = 98%, ISR = 0.57

with only 2.04 components). The seed divergence itself is a finding: identical training setups produce qualitatively different failure modes, making single-seed evaluation unreliable.

5.2 SFT Baseline Comparison

To confirm that structural degeneration is an RL optimization artifact, we compare GRPO against supervised fine-tuning (SFT) on the same data.

Table 2: SFT vs. GRPO variants (FOL). SFT preserves structure but achieves lower prove rate. GRPO exec-only maximizes PR at the cost of structure.

Method	PR	ISR	Prov-only ISR	Genuine
Pretrained	0.12	0.06	0.64	50%
SFT	0.48	0.24	0.50	38%
GRPO exec	0.93	0.12	0.13	18%
GRPO β=0.1	0.81	0.34	0.42	26%

SFT achieves proved-only ISR of 0.50, close to the pretrained model's 0.64, confirming that SFT replicates training data structure without the optimization pressure that drives hacking. However, SFT's prove rate (0.48) is far below GRPO (0.93), because SFT cannot discover effective proving strategies beyond the training distribution. This establishes that reward hacking is an optimization artifact: SFT does not hack because it does not optimize.

5.3 O(1) vs. O(N) Computational Ablation

We evaluated a fast $O(1)$ approximation of ISR that tests only the first component, compared against the full $O(N)$ LOO ablation.

The $O(1)$ approximation fails because hacking patterns (e.g., conclusion embedding) do not necessarily place the exploited component at position 1. The correlation between $O(1)$ and $O(N)$ ISR is only ρ=0.18, rendering the fast approximation informationally equivalent to random noise for training. The $O(N)$ computation is therefore a necessary cost of the STA protocol; the 1.62× speedup of $O(1)$ comes at ∼90% information loss.

Table 3: $O(1)$ vs. $O(N)$ ISR computation. The $O(1)$ approximation produces near-random signals (ρ=0.18), while being 1.62× faster.

Method	PR	ISR	Comp.	Time	ρ
$O(1)$ fast	0.91	0.03/0.11†	9.2	42m	0.18
$O(N)$ full	0.93	0.12	8.8	68m	1.00
Exec-only	0.93	0.12	—	35m	N/A

†fast ISR / true ISR when evaluated with full ablation.

6 Claim 2: ISR as Reward Signal Exhibits Goodhart's Law

6.1 ISR β=0.1: Three-Seed Replication

We train GRPO with the ISR bonus (Eq. 3, β=0.1) across three seeds.

Table 4: ISR β=0.1 three-seed replication (FOL). All seeds converge to a consistent region with no tautological collapse.

Seed	PR	ISR	Prov-only ISR	Comp.	Taut.
s42	0.81	0.337	0.415	5.58	No
s43	0.77	0.305	0.396	5.82	No
s44	0.75	0.282	0.376	6.08	No
Mean	0.777	0.308	0.396	5.83	—
±std	±0.030	±0.028	±0.020	±0.25	—

Unlike exec-only GRPO, where three seeds produce three qualitatively different failure modes, ISR β=0.1 converges all seeds to a consistent region (ISR 0.28–0.34, PR 0.75–0.81). The low variance (std(ISR) = 0.028, std(PR) = 0.030) indicates that the β=0.1 bonus acts as a *stabilizer*: the mild structural signal regularizes training dynamics, preventing seed-dependent collapse.

6.2 Random β=0.1: Matched Control

To isolate the effect of ISR signal content from variance injection, we train with a matched ran-

dom control: $r = r_{\text{exec}} \cdot (1 + 0.1 \cdot U)$ where $U \sim \text{Uniform}(0, 1)$.

Table 5: Random β=0.1 three-seed control (FOL). PR is maintained but ISR remains at exec-only baseline levels.

Seed	PR	ISR	Prov-only ISR	Comp.
s42	0.79	0.104	0.131	8.46
s43	0.74	0.089	0.120	8.92
s44	0.76	0.118	0.155	8.15
Mean	0.763	0.104	0.135	8.51
±std	±0.025	±0.015	±0.018	±0.39

Random noise at β=0.1 maintains prove rate (0.763, not significantly different from ISR's 0.777; p=0.52) but provides no structural improvement: ISR remains at 0.104, comparable to exec-only (0.123). Component count (8.51) is significantly higher than ISR's (5.83), indicating more redundant components. The random control confirms that β=0.1 variance injection prevents GRPO saturation (maintaining exploration) but does not guide optimization toward structural quality.

6.3 Statistical Comparison: ISR vs. Random

Table 6 presents the three-seed comparison.

Table 6: ISR β=0.1 vs. Random β=0.1 (3-seed matched comparison). ISR signal provides genuine structural guidance with large effect size.

Metric	ISR	Random	Ratio	p
PR	.777±.030	.763±.025	1.02	.52
ISR	.308±.028	.104±.015	**2.96**	**.002**
Prov-only ISR	.396±.020	.135±.018	2.93	.001
Comp. count	5.83±.25	8.51±.39	0.69	.003
Genuine rate	24.0%	16.2%	—	+7.7pp

The ISR/random ratio is 2.96× (95% CI: [1.66, 4.39]; does not cross 1). Cohen's d = 9.6, far exceeding the 0.8 threshold for large effects. Despite small n=3, the effect size yields p=0.002 (Welch's t-test, t=11.1, df=3.1). Per-seed ratios are consistent: 3.24 (s42), 3.43 (s43), 2.39 (s44).

Prove rate shows no significant difference (p=0.52), confirming that the ISR improvement is "free"—it does not trade off against proving ability. Genuine rate increases by 7.7 percentage points (range: 7.0–8.2pp across seeds).

6.4 Goodhart Collapse at β=0.3

Increasing the ISR bonus strength to β=0.3 triggers Goodhart collapse.

Table 7: Goodhart's Law: full β dose–response (FOL, s42). The transition from useful signal to harmful optimization target occurs between β=0.1 and β=0.3.

β	Signal	PR	ISR	Phase
0	exec-only	0.93	0.123	baseline
0.1	ISR	**0.81**	**0.337**	pre-Goodhart
0.1	random	0.79	0.104	control
0.3	ISR	0.18	0.026	post-Goodhart
0.3	random	0.48	0.045	control

At β=0.3, ISR reward destroys prove rate (0.18), performing 2.7× worse than matched random noise (0.48). This is not merely "diminishing returns"; ISR at high dose is *actively harmful*, with prove rate below random noise. The transition is sharp: between β=0.1 (sweet spot) and β=0.3 (collapse), the optimization crosses from the regime where ISR correlates with quality to the regime where the model exploits ISR as a metric.

This constitutes a textbook Goodhart effect: ISR ceases to be a good measure of structural quality when it becomes the optimization target at sufficient intensity.

6.5 Curriculum Ablation

We test whether curriculum scheduling (gradually increasing ISR weight during training) can expand the sweet spot.

Under the multiplicative formula (Eq. 3), fixed β=0.1 is already optimal. Curriculum scheduling adds complexity without improvement because the multiplicative formula ensures ISR is always a bonus (≥ 0), eliminating the early-penalty problem that curricula are designed

Table 8: Curriculum ablation for ISR reward scheduling. Fixed β=0.1 outperforms both sigmoid and linear curricula under the multiplicative formula.

Schedule	Formula	PR	ISR
Fixed β=0.1	multiplicative	**0.81**	**0.34**
Sigmoid (mid=200)	convex	0.77	0.23
Linear (mid=200)	convex	0.73	0.19
Fixed β=0.3	multiplicative	0.18	0.03

to address. Sigmoid slightly outperforms linear (ISR 0.23 vs. 0.19) due to its steeper mid-training ramp, but neither matches fixed β=0.1. Linear midpoint sensitivity analysis shows that midpoint=100 (too early, PR=0.70), midpoint=200 (PR=0.73), and midpoint=300 (too late, PR=0.75, ISR=0.15) all underperform.

7 Claim 3: Cross-Logic Generality

7.1 Prolog: Exec-Only GRPO

We apply exec-only GRPO to Prolog formalization (SWI-Prolog solver) across three seeds.

Table 9: Prolog exec-only GRPO (3 seeds). High prove rate with low ISR replicates the FOL hacking pattern. Each seed develops a different dominant hacking mode.

Seed	PR	ISR	Comp.	Hacking	Gen.
s42	0.85	0.08	7.4	concl. emb.	16%
s43	0.79	0.13	6.5	rule dupl.	21%
s44	0.88	0.05	8.3	cut tautol.	12%
Mean	0.84	0.087	7.4	—	16.3%
±std	±.046	±.040	±0.9	—	—

The pattern mirrors FOL: high PR (0.84) with low ISR (0.087), and three seeds producing three different dominant hacking modes. Hacking rate is 83.7%, comparable to FOL's ∼82%. Prolog-specific hacking includes cut tautologies (`!` to force success) and rule duplication (repeating clauses).

7.2 Prolog: Exec+STA with ISR β=0.1

The β=0.1 ISR bonus in Prolog produces results parallel to FOL.

Table 10: Prolog exec+STA β=0.1 (3 seeds). ISR improvement of 2.99× over exec-only replicates the FOL sweet spot.

Seed	PR	ISR	Comp.	Genuine
s42	0.78	0.28	5.1	28%
s43	0.73	0.24	5.5	24%
s44	0.76	0.26	5.3	22%
Mean	0.757	0.260	5.3	24.7%
±std	±.025	±.020	±0.2	—

ISR improves from 0.087 (exec-only) to 0.260 (2.99×), with PR retained at 90% of exec-only levels (0.757/0.84). Genuine rate increases from 16.3% to 24.7% (+8.4pp), closely matching FOL's +7.7pp delta. The cross-logic consistency of the β=0.1 effect (2.96× in FOL, 2.99× in Prolog) suggests the sweet spot is a general property of LOO-based structural rewards in GRPO, not logic-specific.

Table 11: Cross-logic comparison of ISR β=0.1 effect.

	FOL	Prolog	SMT-LIB
ISR impr. over exec	2.96×	2.99×	2.67×
PR retention	84%	90%	93%
Genuine Δ	+7.7pp	+8.4pp	—

7.3 Cross-Logic Hacking Taxonomy

We classify hacking modes across FOL and Prolog exec-only outputs into a unified taxonomy.

Four modes are shared (Jaccard similarity = 0.667), confirming that hacking strategies arise from GRPO optimization dynamics under binary rewards rather than logic-specific properties. Conclusion embedding dominates in both logics (72.8% in FOL, 62% in Prolog), as it is the most direct shortcut: encoding the conclusion as a premise guarantees proof success with minimal

Table 12: Cross-logic hacking taxonomy. Four of five modes are shared, reflecting common GRPO optimization dynamics rather than logic-specific exploits.

Mode	FOL	Prolog	Mechanism
Concl. embedding	72.8%	62%	Encode conclusion
Repetition/dupl.	✓	12%	Repeat effective pat.
Tautological	✓	8%	Trivial truth path
Complexity w/o util.	3.8%	2%	Unused components
Vacuous truth	✓	—	FOL-only (quantifiers)
Backtrack exploit.	—	5%	Prolog-only

structure. Logic-specific modes reflect unique exploitable features: FOL's quantifiers enable vacuous truth (universally quantified over empty domains), while Prolog's backtracking enables accidental query satisfaction.

7.4 SMT-LIB: Third Logic Validation

We extend STA to SMT-LIB/Z3, a constraint satisfaction paradigm distinct from both FOL derivation and Prolog resolution.

Table 13: SMT-LIB GRPO results (single seed). The exec-only hacking and β=0.1 improvement pattern replicates.

Condition	PR	ISR	Comp.
Exec-only	0.72	0.09	8.1
Exec+STA β=0.1	0.67	0.24	5.4
ISR improvement	—	2.67×	0.67×

Exec-only GRPO on SMT-LIB produces hacking (PR 0.72, ISR 0.09) with patterns including over-constrained assertions and trivial assertions. Three hacking modes are shared with both FOL and Prolog. ISR β=0.1 improves structural quality by 2.67×, consistent with the ∼3× improvement observed in FOL and Prolog. The slightly lower ratio may reflect SMT-LIB's more rigid constraint structure, which limits the reward shaping space.

8 Scaling and Multi-Logic Analysis

8.1 Model Scale Effect

We evaluate whether scaling from 0.5B to 8B parameters alleviates reward hacking.

Table 14: Model scale effect (FOL). Scaling reduces but does not eliminate hacking. ISR β=0.1 remains effective at larger scale with better PR retention.

Scale	Condition	PR	ISR	Gen.	Hack.
0.5B	exec-only	0.93	0.12	18%	82%
	β=0.1	0.81	0.34	26%	74%
8B	exec-only	0.96	0.19	32%	68%
	β=0.1	0.92	0.42	35%	65%

Scaling reduces exec-only hacking rate from 82% to 68%: the 8B model retains more incidental structure due to greater representation capacity (ISR 0.19 vs. 0.12). However, hacking persists at 68%, indicating that the problem is fundamentally about the optimization objective, not model capacity.

ISR β=0.1 at 8B achieves ISR = 0.42 (vs. 0.34 at 0.5B) with PR retention of 95.8% (0.92/0.96), substantially better than 0.5B's 87.1% (0.81/0.93). This suggests that larger models have sufficient capacity to simultaneously optimize for prove rate and structural quality, and that the Goodhart sweet spot may shift rightward (tolerating higher β) at larger scale.

8.2 Multi-Logic Joint Training

We test joint FOL+Prolog training on the 0.5B model.

Joint training on the 0.5B model incurs a capacity penalty: FOL PR drops from 0.93 to 0.72 (−0.21), Prolog PR from 0.84 to 0.68 (−0.16). This is a standard multi-task learning tradeoff at limited model capacity. Crucially, the ISR bonus remains effective in the joint setting (FOL ISR 0.21 vs. 0.10; Prolog ISR 0.18 vs. 0.07), though improvement ratios (2.1×/2.6×) are lower than

Table 15: Multi-logic joint training (0.5B). Capacity bottleneck reduces per-task performance, but ISR signal remains effective.

Condition	FOL PR	FOL ISR	Prolog PR	Prolog ISR
Joint exec	0.72	0.10	0.68	0.07
Joint STA	0.67	0.21	0.63	0.18
Single exec	0.93	0.12	0.84	0.09
Single STA	0.81	0.34	0.76	0.26

single-logic (∼3×), indicating that capacity constraints limit structural improvement ceiling.

8.3 Cross-Logic Transfer

We evaluate whether formalization skills transfer across logics.

Table 16: Cross-logic transfer. Limited transfer (28–38% of in-domain) confirms that formalization is largely syntax-specific.

Source → Target	PR	ISR	vs. in-domain
FOL → Prolog	0.32	0.04	38%
Prolog → FOL	0.28	0.03	30%
FOL → FOL	0.93	0.12	100%
Prolog → Prolog	0.84	0.09	100%

Cross-logic transfer is limited: FOL-trained models achieve only 38% of in-domain Prolog performance, and Prolog-trained models achieve 30% of FOL. Transfer ISR is near-zero (0.03–0.04), indicating that transferred proofs that happen to succeed are structurally degenerate. The 28–32% transfer rate exceeds random baseline (∼12%), suggesting partial transfer of high-level reasoning strategies, but syntax-specific optimization dominates. This result supports the need for per-logic STA instantiation rather than relying on cross-logic transfer.

9 Discussion

ISR as Diagnostic vs. Reward. Our results reveal a dual role for ISR: as a diagnostic metric, it reliably detects reward hacking across logics, model scales, and training horizons. As a reward signal, it occupies a narrow sweet spot (β=0.1) where it provides genuine structural guidance, beyond which Goodhart effects dominate. The sharp Goodhart boundary between β=0.1 and β=0.3 is a practical finding: practitioners should treat ISR primarily as a diagnostic and use it as a reward signal only with extreme caution.

GRPO Saturation Mechanism. As prove rate approaches 1.0, the binary execution reward provides vanishing gradient signal (all outputs in a group receive r=1, eliminating the advantage signal). This saturation creates a "training trap": the model continues to update via KL penalty but loses directional gradient from the reward. The β=0.1 bonus breaks this saturation by providing within-group variance even when all outputs are proved, but the structural content of the signal (ISR vs. random) determines whether the variance redirects toward quality or merely randomizes.

Hacking as Systematic Phenomenon. The cross-logic taxonomy (4 shared modes out of 5) establishes reward hacking as a systematic property of binary-reward GRPO, not a quirk of FOL or specific solver implementations. The dominant mode—conclusion embedding—is a universal shortcut: in any formal system, encoding the conclusion as an axiom guarantees proof success with minimal structural integrity.

Limitations. (1) ISR measures syntactic necessity, not semantic faithfulness; downstream NLI correlation is non-significant. (2) Three seeds provide limited statistical power, though the extreme effect size (d=9.6) compensates. (3) The Goodhart transition between β=0.1 and β=0.3 is coarsely characterized; finer-grained β sweeps would better map the boundary. (4) Joint multi-logic training suffers capacity constraints at 0.5B scale; results at 8B may differ. (5) SMT-LIB and 8B experiments use single seeds.

10 Conclusion

We introduced Solver Trace Ablation (STA), a diagnostic protocol that applies leave-one-out

component necessity testing to detect and characterize reward hacking in GRPO-trained formal reasoners. Our key findings are:

1. ISR detects structural degeneration invisible to prove rate: exec-only GRPO achieves 93% prove rate while ISR collapses from 0.64 to 0.003 over 1000 training steps.

2. ISR as a reward signal exhibits Goodhart's Law with a sharp transition: β=0.1 yields 2.96× structural improvement over matched random control (p=0.002), while β=0.3 is worse than random noise.

3. The STA protocol generalizes across FOL, Prolog, and SMT-LIB, with 4/5 hacking modes shared and consistent ∼3× ISR improvement at β=0.1.

For practitioners, we offer four actionable recommendations: (a) never trust prove rate alone—use ISR and component count as joint diagnostics; (b) single-seed GRPO results are unreliable; (c) ISR reward has a narrow sweet spot (β=0.1); and (d) treat ISR primarily as a diagnostic, not a training signal.

A Hacking Mode Examples

Conclusion Embedding (FOL). The model includes the conclusion as a premise: `all x (P(x) -> Q(x)). Q(a). % conclusion directly stated` All non-conclusion components become unnecessary (ISR ≈ 0).

Conclusion Embedding (Prolog). `answer(true) :- true.` The conclusion is hard-coded; no logical derivation occurs.

Repetition Collapse (FOL). The model repeats effective premises multiple times, inflating component count while ISR drops because duplicates are individually unnecessary.

Rule Duplication (Prolog). The same clause appears multiple times; SWI-Prolog matches all but only the first is needed.

Tautological Shortcuts. FOL: Exploit contradiction explosion ($P \wedge \neg P \rightarrow Q$). Prolog: Use cut (`!`) to force success without genuine derivation.

Complexity Without Utility. Unused predicates or premises added to increase apparent complexity without contributing to the proof.